

Chap3-1 HW: AMD uprof profile

1. Profiling `mat_mul_a`

The main difference between `mat_mul_a` and `mat_mul_b` is the loop order used in matrix multiplication. `mat_mul_a` uses the baseline `(i, j, k)` order, while `mat_mul_b` changes it to `(i, k, j)`. Although both programs compute the same result, this change affects how matrix B is accessed in memory and therefore has a major impact on cache behavior.

[`mat_mul_a.c` showing the `(i, j, k)` loop]

```
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        for (int k = 0; k < N; k++)
```

[`mat_mul_b.c` showing the `(i, k, j)` loop]

```
for (int i = 0; i < N; i++)
    for (int k = 0; k < N; k++)
        for (int j = 0; j < N; j++)
```

This difference is already visible in the runtime. In my test, `mat_mul_a` took about **81.173s**, while `mat_mul_b` took about **14.510 s**, meaning `mat_mul_b` was roughly **5.6× faster**. Since the arithmetic is the same, this strongly suggests that the original bottleneck comes from inefficient memory access rather than computation itself.

[terminal output showing `time ./mat_mul_a` and `time ./mat_mul_b`]

```
william@head:~/ch3-1_hw/HPC-I_mat-mul$ time ./mat_mul_a
time ./mat_mul_b

real    1m21.173s
user    2m42.185s
sys     0m0.072s

real    0m14.510s
user    0m28.927s
sys     0m0.046s
william@head:~/ch3-1_hw/HPC-I_mat-mul$
```

In the hotspot report, `mat_mul_omp_fn.0` accounts for about 187.68 s of CPU time and clearly dominates execution.

10 HOTTEST FUNCTIONS (Sort Event - CPU_TIME)			
FUNCTION	CPU_TIME (seconds)	TOTAL_CPU_TIME (s)	Module
<code>mat_mul_omp_fn.0</code>	187.68	187.68	<code>/home/william/ch3-1_hw/HPC-l_mat-mul/mat_mul_a</code>
<code>libgomp.so.1.0.0!0x25</code>	0.47	0.47	<code>/usr/lib/x86_64-linux-gnu/libgomp.so.1.0.0</code>
<code>libgomp.so.1.0.0!0x25</code>	0.11	0.11	<code>/usr/lib/x86_64-linux-gnu/libgomp.so.1.0.0</code>
<code>libc.so.6!0x1a1bfa</code>	0.03	0.03	<code>/usr/lib/x86_64-linux-gnu/libc.so.6</code>
<code>write</code>	0.01	0.01	<code>/usr/lib/x86_64-linux-gnu/libc.so.6</code>
<code>read</code>	0.01	0.01	<code>/usr/lib/x86_64-linux-gnu/libc.so.6</code>
<code>_IO_file_xsputn</code>	0	0.01	<code>/usr/lib/x86_64-linux-gnu/libc.so.6</code>
<code>_IO_file_write</code>	0	0.01	<code>/usr/lib/x86_64-linux-gnu/libc.so.6</code>
<code>fwrite</code>	0	0.01	<code>/usr/lib/x86_64-linux-gnu/libc.so.6</code>
<code>_IO_fread</code>	0	0.01	<code>/usr/lib/x86_64-linux-gnu/libc.so.6</code>

The data-access report gives a second piece of evidence. For the same function, the profiler reports **CPI = 2.2123** and **%L1_DC_MISSES = 67.9637%**, which indicates a very high L1 data-cache miss rate. This means the kernel has poor cache locality and is repeatedly fetching data inefficiently.

10 HOTTEST FUNCTIONS (Sort Event - CYCLES_NOT_IN_HALT)					
FUNCTION	CYCLES_NOT_IN_HALT	RETIRED_INSTRUCTIONS	CPI	L1_DC_ACCESSES	%L1_DC_MISSES
<code>mat_mul_omp_fn.0</code>	199444	90151	2.2123	343.0799	67.9637
<code>libgomp.so.1.0.0!0x25</code>	559	44	12.7045	2568.1818	0
<code>libgomp.so.1.0.0!0x25</code>	62	2	31	8500	0
<code>libc.so.6!0x1a1bfa</code>	47	1	47	15000	50

Based on these results, the two most reasonable optimization directions are:

1. reorder the loops to improve locality, especially for accesses to matrix B
2. apply blocking or tiling so that reused data stays in cache longer

This is also consistent with the large speedup already observed in `mat_mul_b`.

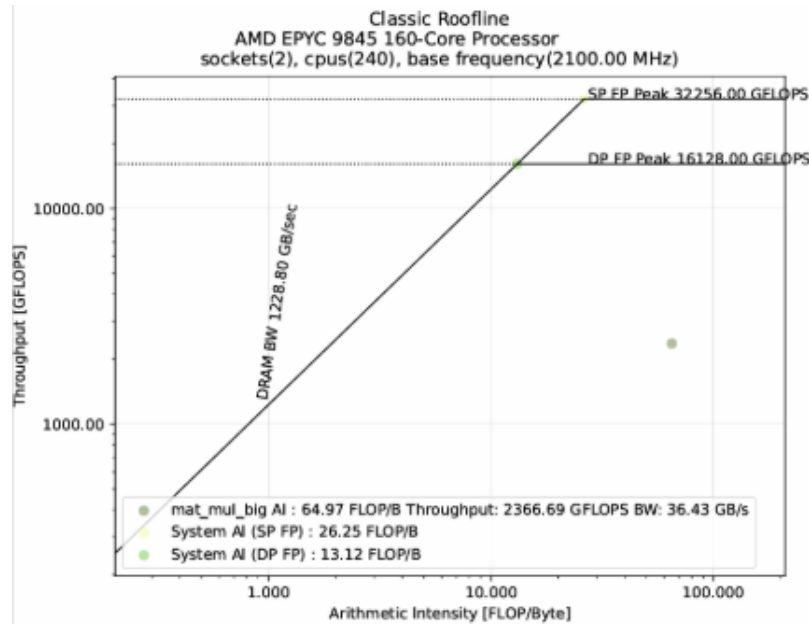
2. Roofline Analysis of `mat_mul_a`

Based on the Roofline model, `mat_mul_a` is **compute-bound** rather than memory-bound.

The main reason is that its arithmetic intensity is about **64.97 FLOP/Byte**, which is much higher than the system ridge point of about **13.12 FLOP/Byte**. Since the program lies to the right of the ridge point, it falls in the compute-bound region of the Roofline model. The measured throughput is about **2366.69 GFLOPS**, which is still far below the peak double-precision

performance, so the program is limited more by how efficiently computation is executed than by DRAM bandwidth.

[Roofline figure for `mat_mul_a`]



This conclusion is not fully consistent with the intuition from Task 1. In Task 1, the profiling results suggested a strong memory-locality problem: `mat_mul_a` had poor cache behavior, a high L1 data-cache miss rate, and a large speedup when the loop order was changed in `mat_mul_b`. That naturally gave the impression that memory access was the main bottleneck.

However, these two results can still be explained together. Task 1 showed that the code has **inefficient cache behavior**, while the Roofline model shows that the program is **not limited by main-memory bandwidth** at the system level. In other words, memory access patterns still matter a lot, but the program is not fundamentally DRAM-bandwidth-bound on this machine. Instead, it is better described as compute-bound overall while still losing performance because of poor cache efficiency.

So, the final interpretation is that the Roofline model classifies `mat_mul_a` as compute-bound, but the profiling from Task 1 remains useful because it explains why the program still performs far below the compute roof.

3. `mat_mul_b` with Different `OMP_NUM_THREADS`

I tested `mat_mul_b` with three OpenMP thread counts: 1, 2, and 4.

The program scales almost ideally from 1 thread to 2 threads, but there is essentially no improvement from 2 threads to 4 threads.

[terminal output showing runtimes for 1, 2, and 4 threads]

```
william@head:~/ch3-1_hw/HPC-I_mat-mul$ export OMP_NUM_THREADS=1
time ./mat_mul_b

export OMP_NUM_THREADS=2
time ./mat_mul_b

export OMP_NUM_THREADS=4
time ./mat_mul_b

real    0m29.378s
user    0m29.338s
sys     0m0.038s

real    0m14.478s
user    0m28.869s
sys     0m0.041s

real    0m14.390s
user    0m28.551s
sys     0m0.046s
william@head:~/ch3-1_hw/HPC-I_mat-mul$
```

OpenMP trace summary and thread summary for OMP_NUM_THREADS = 1

APPLICATION PERFORMANCE SNAPSHOT		
Number Of Threads	1	
.. OPENMP SUMMARY		
.. Idle Time	0 seconds	

THREAD SUMMARY		
THREAD	CPU_TIME (s)	THREAD ID
mat_mul_b (PID:6885)	29.021	6885

PARALLEL EFFICIENCY REPORT	
Parallel Inefficiency	0
.. MPI Load Balance Ineffi	0
.. MPI Communication In	0
.. Serial Region Inefficien	0
.. OpenMP Potential Gair	0
.... Potential Gain:Sync	0
.... Potential Gain:Other	0

OpenMP trace summary and thread summary for OMP_NUM_THREADS = 2

APPLICATION PERFORMANCE SNAPSHOT		
Number Of Threads	2	
.. OPENMP SUMMARY		
.. Idle Time	0 seconds	
THREAD SUMMARY		
THREAD	CPU_TIME (s)	THREAD ID
mat_mul_b (PID:6902)	14.679	6902
Thread-6906 (PID:6906)	14.645	6906

OpenMP trace summary and thread summary for OMP_NUM_THREADS = 4

APPLICATION PERFORMANCE SNAPSHOT		
Number Of Threads	4	
.. OPENMP SUMMARY		
.. Idle Time	0 seconds	

THREAD SUMMARY		
THREAD	CPU_TIME (s)	THREAD ID
mat_mul_b (PID:6918)	7.261	6918
Thread-6923 (PID:6923)	7.258	6923
Thread-6922 (PID:6922)	7.23	6922
Thread-6924 (PID:6924)	7.218	6924

The OpenMP trace reports confirm the runs used 1, 2, and 4 threads, and the thread summaries show that the work is distributed across the available threads. The runtime improves strongly from 1 thread to 2 threads, but shows almost no further improvement at 4 threads. The parallel-efficiency report also shows no meaningful extra gain from increasing the thread count beyond the available CPU resources.

This behavior matches **Amdahl's Law**. The speedup from parallelization is limited by the serial portion of the program and by runtime overhead. In addition, this VM only exposes 2 CPUs, so using 4 threads likely causes oversubscription instead of real additional parallelism. That explains why the 4-thread case is almost identical to the 2-thread case.

4. HPL Benchmark with Three Configurations

For the HPL benchmark, I kept $N = 1000$, $P = 2$, and $Q = 2$ fixed, and varied only the blocking factor NB . This makes the comparison controlled and allows the effect of NB to be observed more clearly.

The three tested configurations were:

- **Config 1:** $NB = 64$, $P = 2$, $Q = 2$
- **Config 2:** $NB = 128$, $P = 2$, $Q = 2$
- **Config 3:** $NB = 256$, $P = 2$, $Q = 2$

All HPL runs used 4 MPI processes arranged as $P \times Q = 2 \times 2$

4.1 Config 1: $NB = 64$

The first configuration completed successfully, and all 18 tests passed the residual checks.

[NB=64]

```
N      :    1000
NB     :     64
PMAP   : Row-major process mapping
P      :      2
Q      :      2
PFACT  : Left   Crout   Right
NBMIN  :      2      4
NDIV   :      2
RFACT  : Left   Crout   Right
BCAST  : 1ring
DEPTH  :      0
SWAP   : Mix (threshold = 64)
L1     : transposed form
U      : transposed form
EQUIL  : yes
ALIGN  : 8 double precision words
```

[bottom section showing Finished 18 tests and PASSED]

```
Finished      18 tests with the following results:
               18 tests completed and passed residual checks,
                0 tests completed and failed residual checks,
                0 tests skipped because of illegal input values.
```

```
-----
End of Tests.
=====
```

For profiling, I collected hotspots, assess, and pipeline-utility results. The default hotspots signal conflicted with the MPI workflow, so I reran it with `--profiling-signal 38`, which produced a valid report with actual HPL-related functions such as `rpcc`, `exec_blas_async_wait`, `blas_thread_server`, and `HPL_lmul`.

[hpc_hw/hpl_profiles/cfg1/mpi_hotspots_sig38/AMDuProf-run_hpl_cfg1-
Hotspots_Apr-19-2026_21-25-45/report.csv]

PROFILE DETAILS			
Profile Session Type:	Hotspots		
Profile Scope:	Single Application		
CPU Mask:	0, 1		
CPU Affinity Mask:	0, 1		
Profile Start Time:	Sun Apr 19 21:25:45 2026		
Profile End Time:	Sun Apr 19 21:26:09 2026		
Profile Duration:	23.676 seconds		
Data Folder:	/home/william/hpl_profiles/cfg1/mpi_hotspots_sig38/AMDuProf-run		
Virtual Machine:	Yes		
Hypervisor Vendor:	Linux KVM		
Timer Interval:	10ms		
Profiling Signal:	38		
Call Stack Sampling:	TRUE		
Call Stack Overcome fram	FALSE		
Call Stack Unwind Depth:	32		
Selected View:	Time based hotspots		
MONITORED EVENTS			
TBP Events:	Name	Interval	Description
	CPU_TIME	10.000 millis	CPU Time is the time in which the CPU co
DERIVED METRICS			
Metric Name	Expression	Description	
TOTAL_CPU_TIME (s)	CPU_TIME	The CPU time spent by a function and its children.	
APPLICATION PERFORMANCE SNAPSHOT			
Number Of Threads	11		
10 HOTTEST FUNCTIONS (Sort Event - CPU_TIME)			
FUNCTION	CPU_TIME (s)	TOTAL_CPU_	Module
rpcc	7.17	7.17	/home/william/hpl/bin/linux/xhpl
epoll_wait	4.22	4.33	/usr/lib/x86_64-linux-gnu/libc.so.6
exec_blas_async_wait	1.65	1.65	/home/william/hpl/bin/linux/xhpl
inner_thread	0.97	0.98	/home/william/hpl/bin/linux/xhpl
blas_thread_server	0.79	8.36	/home/william/hpl/bin/linux/xhpl
HPL_tmul	0.57	0.57	/home/william/hpl/bin/linux/xhpl

[hpc_hw/hpl_profiles/cfg1/assess/AMDuProf-run_hpl_cfg1-EBP_Apr-19-2026_21-24-
05/report.csv]

PERFORMANCE ANALYSIS REPORT			
EXECUTION			
Target Path:	./run_hpl_cfg1.sh		
Command Line Arguments:			
Working Directory:	/home/william/hpl/bin/linux		
Environment Variables:			
CPU Details:	Family(0x19), Model(0x11), Number of C		
Operating System:	Linux Ubuntu 24.04.4 LTS-64 Kernel:6.8.0		
Collect Command:	./AMDuProfCLI collect --config assess -g		
Report Command:	./AMDuProfCLI report --detail -i /home/w		
PROFILE DETAILS			
Profile Session Type:	Assess Performance		
Profile Scope:	Single Application		
CPU Mask:	0, 1		
CPU Affinity Mask:	0, 1		
Profile Start Time:	Sun Apr 19 21:24:05 2026		
Profile End Time:	Sun Apr 19 21:24:20 2026		
Profile Duration:	15.602 seconds		
Data Folder:	/home/william/hpl_profiles/cfg1/assess/		
Virtual Machine:	Yes		
Hypervisor Vendor:	Linux KVM		
Call Stack Sampling:	TRUE		
Call Stack Overcome frame-po	FALSE		
Call Stack Size(Bytes):	1024		
Selected View:	Overall Assessment		

[hpc_hw/hpl_profiles/cfg1/pipeline_util.csv]

CORE METRICS					
System (Aggregated)					
Total_Dispat	SMT_Dispc	Frontend_Bound	Bad_Specula	Backend_Boi	Retiring
1.186E+11	0	30.85	0.81	28.87	44.18
1.5663E+11	0	34.62	0.8	18	37.38
1.5675E+11	0	32.9	0.61	27.08	41.64
1.567E+11	0	33.14	1.08	25.91	39.54
1.5574E+11	0	28.53	0.51	23.32	42.9
1.5744E+11	0	33.47	0.87	21.15	40.26
1.5657E+11	0	37.97	0.71	24.86	39.13
1.5692E+11	0	37.01	0.55	28.07	40.3
1.5649E+11	0	36.11	0.74	26.33	39.03
1.559E+11	0	36.83	0.75	23.45	40.92
1.5615E+11	0	28.08	0.93	25.45	40.14
1.5583E+11	0	38.47	0.86	22.13	39.73
1.5467E+11	0	35.35	0.61	26.42	43.58

The best visible performance in this configuration was about **2.6942 GFLOPS**.

4.2 Config 2: NB = 128

The second configuration also completed successfully, and all 18 tests passed the residual checks.

[NB=128]

The following parameter values will be used:

```
N      :    1000
NB     :    128
PMAP   : Row-major process mapping
P      :      2
Q      :      2
PFACT  : Left  Crout  Right
NBMIN  :      2      4
NDIV   :      2
RFACT  : Left  Crout  Right
BCAST  : 1ring
DEPTH  :      0
SWAP   : Mix (threshold = 64)
L1     : transposed form
U      : transposed form
EQUIL  : yes
ALIGN  : 8 double precision words
```

[bottom section showing Finished 18 tests and PASSED]

```
-----
Finished      18 tests with the following results:
               18 tests completed and passed residual checks,
               0 tests completed and failed residual checks,
               0 tests skipped because of illegal input values.
-----
```

```
End of Tests.
=====
```

The corrected hotspots profile again used `--profiling-signal 38` and successfully captured HPL-related functions.

[hpc_hw/hpl_profiles/cfg2/mpi_hotspots_sig38/AMDuProf-run_hpl_cfg1-Hotspots_Apr-19-2026_21-29-44/report.csv]

PROFILE DETAILS			
Profile Session Type:	Hotspots		
Profile Scope:	Single Application		
CPU Mask:	0, 1		
CPU Affinity Mask:	0, 1		
Profile Start Time:	Sun Apr 19 21:29:44 2026		
Profile End Time:	Sun Apr 19 21:30:06 2026		
Profile Duration:	21.922 seconds		
Data Folder:	/home/william/hpl_profiles/cfg2/mpi_hotspots_sig38/AMDuProf-run_hpl_cfg1-Hotspots_Apr-19-2026_21-29		
Virtual Machine:	Yes		
Hypervisor Vendor:	Linux KVM		
Timer Interval:	10ms		
Profiling Signal:	38		
Call Stack Sampling:	TRUE		
Call Stack Overcome fr	FALSE		
Call Stack Unwind Dep	32		
Selected View:	Time based hotspots		
MONITORED EVENTS			
TBP Events:	Name	Interval	Description
	CPU_TIME	10.000 millis	CPU Time is the time in which the CPU core is executing the code.
DERIVED METRICS			
Metric Name	Expression	Description	
TOTAL_CPU_TIME (s)	CPU_TIME	The CPU time spent by a function and its children.	
APPLICATION PERFORMANCE SNAPSHOT			
Number Of Threads	11		
10 HOTTEST FUNCTIONS (Sort Event - CPU_TIME)			
FUNCTION	CPU_TIME (s)	TOTAL_CPU_	Module
rpcc	6.52	6.52	/home/william/hpl/bin/linux/xhpl
epoll_wait	4	4.09	/usr/lib/x86_64-linux-gnu/libc.so.6
exec_blas_async_wait	1.19	1.19	/home/william/hpl/bin/linux/xhpl
blas_thread_server	0.6	7.3	/home/william/hpl/bin/linux/xhpl
inner_thread	0.59	0.61	/home/william/hpl/bin/linux/xhpl
HPL_lmul	0.52	0.52	/home/william/hpl/bin/linux/xhpl

[hpc_hw/hpl_profiles/cfg2/assess_fixed/AMDuProf-run_hpl_cfg1-EBP_Apr-19-2026_21-40-45/report.csv]

PROFILE DETAILS			
Profile Session Type:	Assess Performance		
Profile Scope:	Single Application		
CPU Mask:	0, 1		
CPU Affinity Mask:	0, 1		
Profile Start Time:	Sun Apr 19 21:40:45 2026		
Profile End Time:	Sun Apr 19 21:41:01 2026		
Profile Duration:	15.605 seconds		
Data Folder:	/home/william/hpl_profiles/cfg2/assess_		
Virtual Machine:	Yes		
Hypervisor Vendor:	Linux KVM		
Call Stack Sampling:	TRUE		
Call Stack Overcome frar	FALSE		
Call Stack Size(Bytes):	1024		
Selected View:	Overall Assessment		

[hpc_hw/hpl_profiles/cfg2/pipeline_util.csv]

CORE METRICS					
System (Aggregated)					
Total_Dispat	SMT_Dispatch	Frontend_Bo	Bad_Specula	Backend_Bo	Retiring
6.6123E+10	0	33.15	0.94	25.97	41.22
1.5345E+11	0	27.37	1.36	22.47	40.06
1.5276E+11	0	37.81	0.7	27.03	40.42
1.5357E+11	0	37.47	0.73	19.11	40.28
1.539E+11	0	38.46	0.65	22.87	38.91
1.5416E+11	0	35.23	0.77	21.05	39.44
1.5436E+11	0	32.69	0.6	20.75	39.35
1.5436E+11	0	33.67	0.74	27.4	38.82
1.5313E+11	0	34.14	0.76	32.6	38.05
1.5465E+11	0	36.83	0.83	24.27	38.09
1.5298E+11	0	28.51	0.85	27.04	39.43

The best visible performance in Config 2 was about **2.1315 GFLOPS**, which is noticeably higher than Config 1.

4.3 Config 3: NB = 256

The third configuration also ran successfully, and all 18 tests passed the residual checks.

[NB=256]

```

The following parameter values will be used:
N      : 1000
NB     : 256
PMAP   : Row-major process mapping
P      : 2
Q      : 2
PFACT  : Left  Crout  Right
NBMIN  : 2      4
NDIV   : 2
RFACT  : Left  Crout  Right
BCAST  : 1ring
DEPTH  : 0
SWAP   : Mix (threshold = 64)
L1     : transposed form
U      : transposed form
EQUIL  : yes
ALIGN  : 8 double precision words

```

[bottom section showing Finished 18 tests and PASSED]

```
=====
Finished      18 tests with the following results:
              18 tests completed and passed residual checks,
              0 tests completed and failed residual checks,
              0 tests skipped because of illegal input values.
=====
```

```
End of Tests.
=====
```

Hotspots, assess, and pipeline-utility profiles were collected again for this configuration. The hotspots report shows that the signal-38 workaround continued to work correctly and still captured valid HPL/OpenMPI-related functions.

[hpc_hw/hpl_profiles/cfg3/mpi_hotspots_sig38/AMDuProf-run_hpl_cfg1-Hotspots_Apr-19-2026_21-35-08/report.csv]

PROFILE DETAILS			
Profile Sessic Hotspots			
Profile Scope Single Application			
CPU Mask:	0, 1		
CPU Affinity	0, 1		
Profile Start Time	Sun Apr 19 21:35:08 2026		
Profile End Time	Sun Apr 19 21:35:29 2026		
Profile Duration	21.656 seconds		
Data Folder:	/home/william/hpl_profiles/cfg3/mpi_hotspots_sig38/AMDuProf-run_hpl_cfg1-Ho		
Virtual Mach	Yes		
Hypervisor	Linux KVM		
Timer Interval	10ms		
Profiling Signal	38		
Call Stack Saved	TRUE		
Call Stack Overflow	FALSE		
Call Stack Unroll	32		
Selected View	Time based hotspots		
MONITORED EVENTS			
TBP Events:	Name	Interval	Description
	CPU_TIME	10.000 millis	CPU Time is the time in which the CPU core is executin
DERIVED METRICS			
Metric Name	Expression	Description	
TOTAL_CPU_	CPU_TIME	The CPU time spent by a function and its children.	
APPLICATION PERFORMANCE SNAPSHOT			
Number Of Tr	11		
10 HOTTEST FUNCTIONS (Sort Event - CPU_TIME)			
FUNCTION	CPU_TIME (s)	TOTAL_CPU_	Module
rpcc	4.74	4.74	/home/william/hpl/bin/linux/xhpl
epoll_wait	4.31	4.38	/usr/lib/x86_64-linux-gnu/libc.so.6
exec_blas_as	0.99	0.99	/home/william/hpl/bin/linux/xhpl
inner_thread	0.57	0.6	/home/william/hpl/bin/linux/xhpl
blas_thread_	0.47	5.53	/home/william/hpl/bin/linux/xhpl
writerv	0.42	0.42	/usr/lib/x86_64-linux-gnu/libc.so.6
HPL_lmul	0.39	0.39	/home/william/hpl/bin/linux/xhpl

[hpc_hw/hpl_profiles/cfg3/assess/AMDuProf-run_hpl_cfg1-EBP_Apr-19-2026_21-36-04/report.csv]

PROFILE DETAILS			
Profile Session T	Assess Performance		
Profile Scope:	Single Application		
CPU Mask:	0, 1		
CPU Affinity Mas	0, 1		
Profile Start Tim	Sun Apr 19 21:36:04 2026		
Profile End Time	Sun Apr 19 21:36:18 2026		
Profile Duration	13.596 seconds		
Data Folder:	/home/william/hpl_profiles/cfg3/assess/		
Virtual Machine	Yes		
Hypervisor Venc	Linux KVM		
Call Stack Samp	TRUE		
Call Stack Over	FALSE		
Call Stack Size(	1024		
Selected View:	Overall Assessment		

[hpc_hw/hpl_profiles/cfg3/pipeline_util.csv]

CORE METRICS					
System (Aggregated)					
Total_Dispat	SMT_Dispc	Frontend_Bo	Bad_Specula	Backend_Bo	Retiring
9.8656E+10	0	38.95	0.9	21.1	36.64
1.5177E+11	0	34.83	1.27	22.44	38.81
1.5583E+11	0	39.45	0.99	24.12	34.89
1.5582E+11	0	40.5	0.82	24.03	34.24
1.5313E+11	0	40.43	1.14	25.15	36.01
1.5397E+11	0	41.28	1.06	28.46	34.61
1.5126E+11	0	38.94	1.08	26.73	34.17
1.5461E+11	0	39.68	0.97	21.6	36.39
1.5216E+11	0	37.49	1.1	31.16	36.75
1.5183E+11	0	36.24	1.15	28.88	35.14

The best visible performance in Config 3 was about **2.3626 GFLOPS**.

4.4 Comparison of HPL Results

The best visible GFLOPS values from the three configurations are summarized below:

Config NB Best visible GFLOPS

1	64	2.6942
2	128	2.1315
3	256	2.3626

These results show that, in these runs, $NB = 64$ produced the highest visible HPL performance, followed by $NB = 256$, while $NB = 128$ was the lowest among the three tested configurations.

5. Conclusion

The profiling results show that the main issue in `mat_mul_a` is poor memory locality. The program spends almost all of its time in the matrix multiplication kernel, and the data-access profile shows a very high L1 data-cache miss rate. This explains why simply changing the loop order in `mat_mul_b` produces a large speedup. The most natural optimization directions are therefore loop reordering and blocking.

The OpenMP experiments show that `mat_mul_b` scales well from 1 thread to 2 threads, but not beyond that. This is consistent with Amdahl's Law and with the fact that the VM only provides 2 visible CPUs.

For the HPL benchmark, comparing three values of NB while keeping the other parameters fixed shows that $NB = 64$ gave the best observed HPL performance on this system among the three tested configurations. Overall, the profiler results and benchmark outputs consistently show that memory behavior and parameter choice have a strong impact on performance.